



Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales

Electrónica Digital III
Trabajo Practico Integrador: Grabador-Reproductor de voz

Docente:
Ing. Emiliano Elvio Migliore

Autores:
Matías Lucas Morales Robles
José Ignacio Torres
Esteban Samuel Suarez

Índice

1. Proyecto	2
2. Descripción del SEP	3
2.1. Adquisición de la señal de sonido (SSE 2.1 y SSE 2.2)	3
2.2. Digitalización y procesamiento (SSEP 2.3 – LPC1769)	3
2.3. Comunicación externa (SSE 2.4 – Serial to USB)	3
2.4. Interfaz de usuario y visualización (SSE 3.1 y SSE 3.2)	3
2.5. Reproducción de la señal (SSE 1.2 y SSE 1.1)	3
2.6. SSE 3.3 — Señalización óptica (LEDs del sistema)	4
2.7. SSE 3.4 — Variación de velocidad de reproducción	4
2.8. SSE 3.5 — Pulsador para iniciar grabación (EINT0)	4
2.9. SSE 3.6 — Pulsador para detener o reanudar reproducción (EINT1)	4
2.10. Interacción general del usuario	4
3. Desarrollo de Hardware	6
3.1. Circuito de los SSE de la Etapa de Amplificación y el Parlante	6
3.2. Circuito de los SSE del Micrófono Electret y de la Etapa de Amplificación y Offset	7
3.3. Circuito del SSE de Señalización Óptica	8
3.4. Circuito del SSE de Variación de velocidad de reproducción	9
3.5. Circuito de los SSE de control de grabación y reproducción	10
4. Desarrollo de Firmware	11
4.1. Firmware del SSE de visualización de la señal	11
4.2. Firmware del proceso de adquisición de la señal de audio	13
4.3. Firmware del SSE de Comunicación de datos	14
4.4. Firmware del SSE de Señalización óptica	15
4.5. Firmware del SSE de variación de velocidad	16
4.6. Firmware del SSE de comienzo de grabación	16
4.7. Firmware del SSE para reanudar y detener la reproducción	17
5. Pruebas de sistema	18
6. Conclusiones	20
7. Bibliografía y referencias	20
8. Anexo	21
8.1. Circuito esquemático completo	21
8.2. Firmware del SEP	21
8.3. Lista de materiales	27

1. Proyecto

El presente trabajo práctico integrador describe la implementación de un *grabador-reproductor de voz*. El mismo se basa en un SSEP LPC1769 como sistema microcontrolador, el cual adquiere la señal analógica de sonido proveniente de un micrófono (previamente preprocesada) para luego reproducirla analógicamente en un parlante y, además, transmitirla a un sistema de comunicación para su visualización en una PC.

El sistema cuenta con mecanismos para variar el volumen de la señal de audio (mediante un potenciómetro) y la velocidad de reproducción (de forma similar a los audios de WhatsApp).

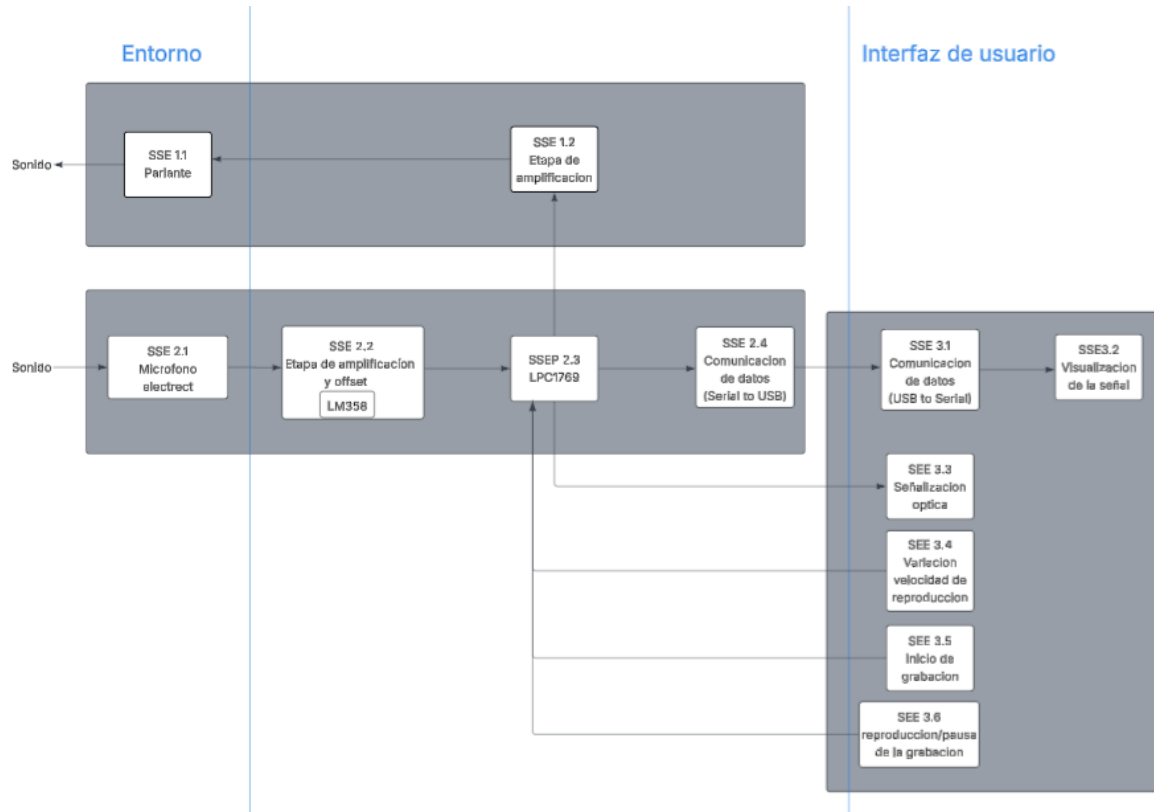


Figura 1: Diagrama en bloques del proyecto

2. Descripción del SEP

El sistema implementado corresponde a un *grabador-reproductor de voz* basado en el microcontrolador **LPC1769**. Su arquitectura general puede dividirse en tres dominios principales, tal como se observa en el diagrama de bloques: (1) Entorno, (2) Sistema Embebido (SSE/SSEP) y (3) Interfaz de Usuario. A continuación se describe en detalle cómo se adquiere, procesa y distribuye la señal de sonido dentro del sistema, y cómo interactúa el usuario con cada etapa.

2.1. Adquisición de la señal de sonido (SSE 2.1 y SSE 2.2)

El proceso inicia en el bloque **SSE 2.1 – Micrófono electret**, ubicado en el dominio del entorno. El micrófono convierte el sonido ambiente en una señal analógica de pequeña amplitud. Esta señal, por sí sola, no es apta para ser digitalizada; por lo tanto, debe acondicionarse.

La salida del micrófono ingresa al bloque **SSE 2.2 – Etapa de amplificación y offset (LM358)**, que realiza:

- **Amplificación:** eleva la amplitud de la señal del micrófono para adaptarla al rango del ADC del LPC1769.
- **Offset:** desplaza la señal para que quede dentro del rango 0–3.3 V, evitando recortes durante la conversión.

La señal acondicionada se entrega al siguiente bloque.

2.2. Digitalización y procesamiento (SSEP 2.3 – LPC1769)

Una vez acondicionada, la señal analógica ingresa al microcontrolador **LPC1769 (SSEP 2.3)**, donde se llevan a cabo varias tareas:

1. **Conversión Analógica–Digital (ADC):** el LPC1769 toma muestras a 8 kHz mediante una interrupción dedicada.
2. **Procesamiento:** el firmware permite:
 - almacenar muestras en un buffer,
 - reproducirlas mediante DMA y DAC,
 - ajustar la velocidad de reproducción en tiempo real,
 - transmitir las por UART para visualizarlas en una PC.
3. **Envío de datos al exterior (UART):** los datos muestreados pueden ser enviados para ser visualizados.

2.3. Comunicación externa (SSE 2.4 – Serial to USB)

El bloque **SSE 2.4 – Comunicación Serial to USB** adapta la señal UART generada por el LPC1769 al estándar USB, permitiendo la transmisión hacia una PC.

2.4. Interfaz de usuario y visualización (SSE 3.1 y SSE 3.2)

La señal digital enviada por UART puede visualizarse a través de:

- **SSE 3.1 – Adaptador USB to Serial**, que convierte la comunicación proveniente del LPC1769,
- **SSE 3.2 – PC**, donde el usuario puede observar la forma de onda adquirida.

Estos componentes conforman la interfaz visual y de monitoreo del sistema.

2.5. Reproducción de la señal (SSE 1.2 y SSE 1.1)

Cuando el sistema se coloca en modo reproducción, las muestras grabadas son enviadas mediante DMA hacia el **DAC del LPC1769**, luego amplificadas por la **SSE 1.2 – Etapa de amplificación**, y finalmente convertidas nuevamente en sonido por **SSE 1.1 – Parlante**.

2.6. SSE 3.3 — Señalización óptica (LEDs del sistema)

El sistema incorpora indicadores luminosos que informan al usuario sobre el estado actual:

- **LED azul** encendido durante la grabación.
- **LED verde** indica reproducción.
- **LED rojo** indica sistema en reposo.

Estos LEDs son gestionados desde las interrupciones del ADC y EINT1, permitiendo retroalimentación inmediata.

2.7. SSE 3.4 — Variación de velocidad de reproducción

La velocidad de reproducción se ajusta mediante un potenciómetro conectado al canal **AD1**. Cuando el sistema **no está grabando**, la interrupción del ADC calcula continuamente el valor:

- Se toma AD1,
- Se aplica la función `map()`,
- Se ajusta el parámetro **DAC_TimeOut**, que determina la frecuencia con la que el DMA actualiza el DAC.

Al modificar **DAC_TimeOut**, cambia la velocidad de reproducción (más rápido o más lento), imitando el comportamiento de reproductores de audio como WhatsApp.

2.8. SSE 3.5 — Pulsador para iniciar grabación (EINT0)

El bloque **SSE 3.5** corresponde al pulsador conectado a **EINT0**, cuya función es:

- iniciar una nueva grabación,
- limpiar el buffer,
- habilitar nuevamente la interrupción del ADC,
- preparar el sistema apagando el DMA para evitar conflictos.

Al presionarse, el sistema ingresa en modo grabación inmediatamente.

2.9. SSE 3.6 — Pulsador para detener o reanudar reproducción (EINT1)

Este pulsador controla el modo reproducción mediante la interrupción **EINT1**. Su comportamiento es:

- Si el audio se está reproduciendo, el pulsador **detiene la reproducción**, detiene el DMA y resetea el DAC.
- Si el sistema está detenido, el pulsador **reanuda la reproducción**, reconfigurando el DMA para transmitir los datos hacia el DAC.

El estado interno **PLAY** evita confusiones, alternando entre iniciar o detener la reproducción.

2.10. Interacción general del usuario

El usuario puede:

- **iniciar una grabación** (SSE 3.5 – pulsador EINT0);
- **detener o reanudar la reproducción** (SSE 3.6 – pulsador EINT1);
- **controlar la velocidad de reproducción** mediante un potenciómetro (SSE 3.4);
- **observar el estado del sistema** mediante LEDs (SSE 3.3);
- **visualizar la señal adquirida** en una PC (SSE 3.1–3.2).

En conjunto, el sistema ofrece una interfaz intuitiva, mientras que el LPC1769 gestiona de forma autónoma la adquisición, procesamiento y reproducción de la señal de audio.



Figura 2: Circuito físico completo

3. Desarrollo de Hardware

3.1. Circuito de los SSE de la Etapa de Amplificación y el Parlante

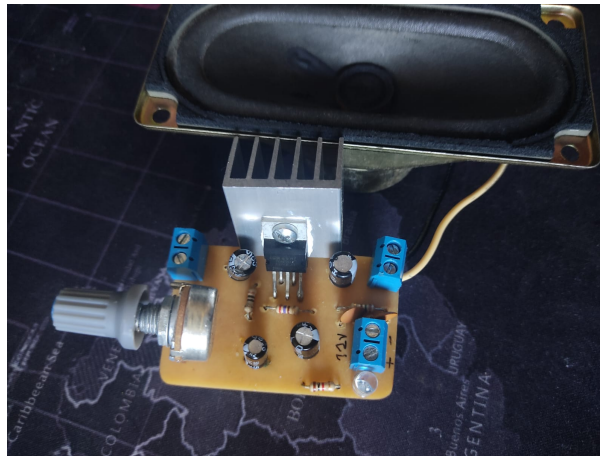


Figura 3: Circuito físico de amplificación para parlante

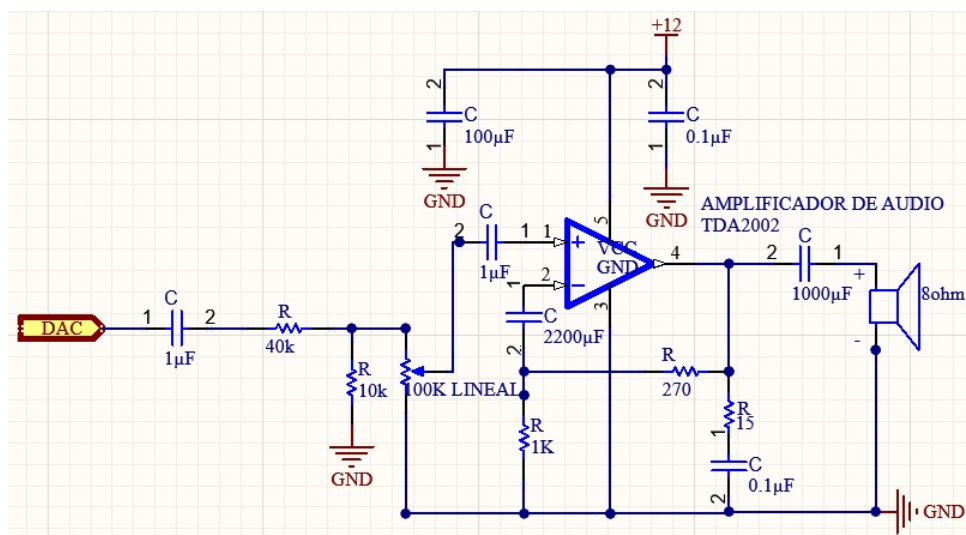


Figura 4: Circuito esquemático de amplificación para parlante

3.2. Circuito de los SSE del Micrófono Electret y de la Etapa de Amplificación y Offset

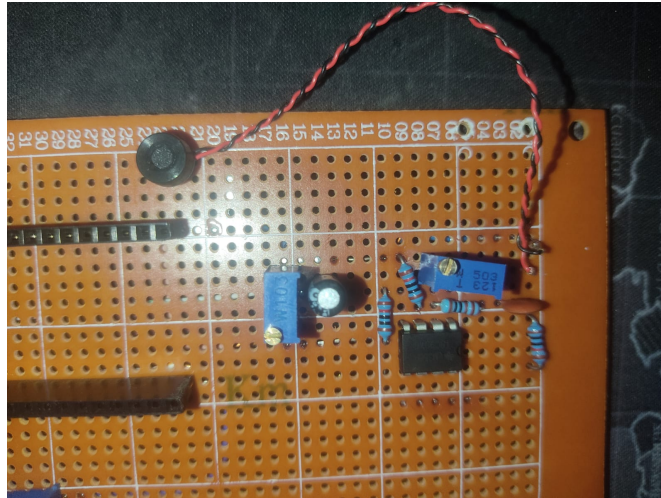


Figura 5: Circuito físico del micrófono

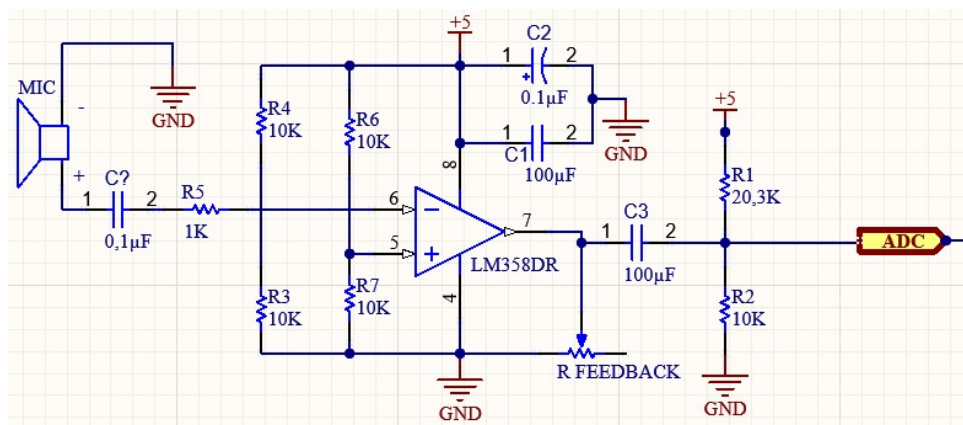


Figura 6: Circuito esquemático del micrófono

3.3. Circuito del SSE de Señalización Óptica

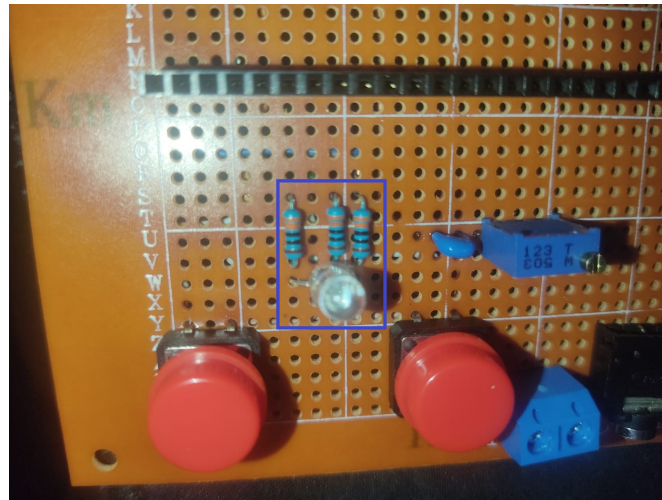


Figura 7: Circuito físico de Señalización óptica

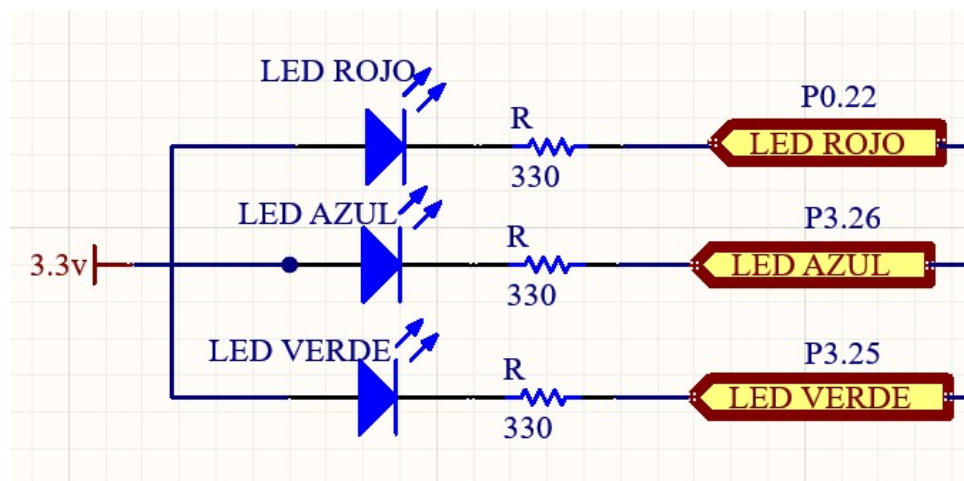


Figura 8: Circuito esquemático de Señalización Óptica

A continuación se muestra el cálculo de las resistencias:

$$R = \frac{3,3\text{ V} - V_{\text{diodo}}}{4,3\text{ mA}} = \frac{3,3\text{ V} - 1,88\text{ V}}{4,3\text{ mA}} \approx 330\ \Omega$$

3.4. Circuito del SSE de Variación de velocidad de reproducción

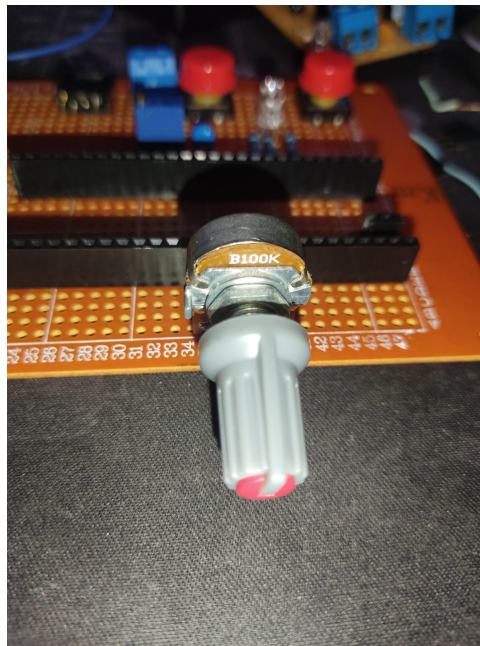


Figura 9: Circuito físico de variación de velocidad de reproducción

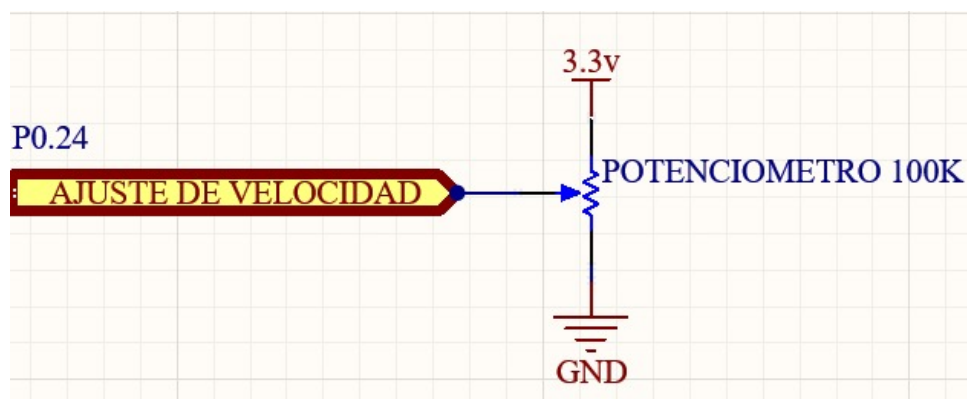


Figura 10: Circuito esquemático de velocidad de reproducción

3.5. Circuito de los SSE de control de grabación y reproducción

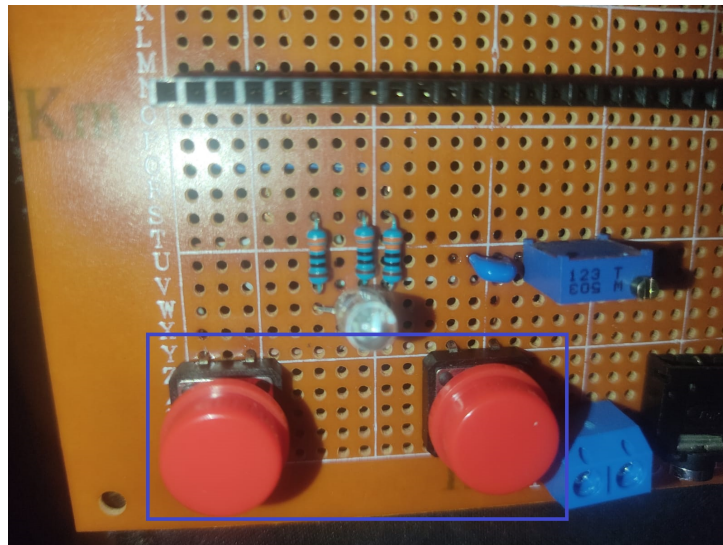


Figura 11: Circuito físico de control de grabación y reproducción

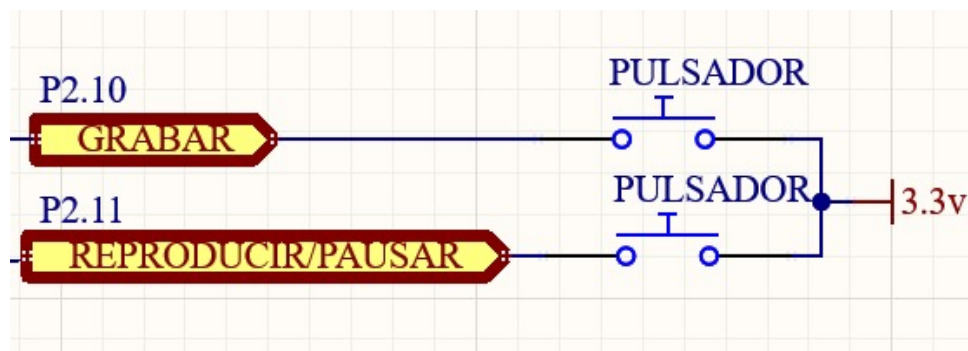


Figura 12: Circuito esquemático de control de grabación y reproducción

4. Desarrollo de Firmware

4.1. Firmware del SSE de visualización de la señal

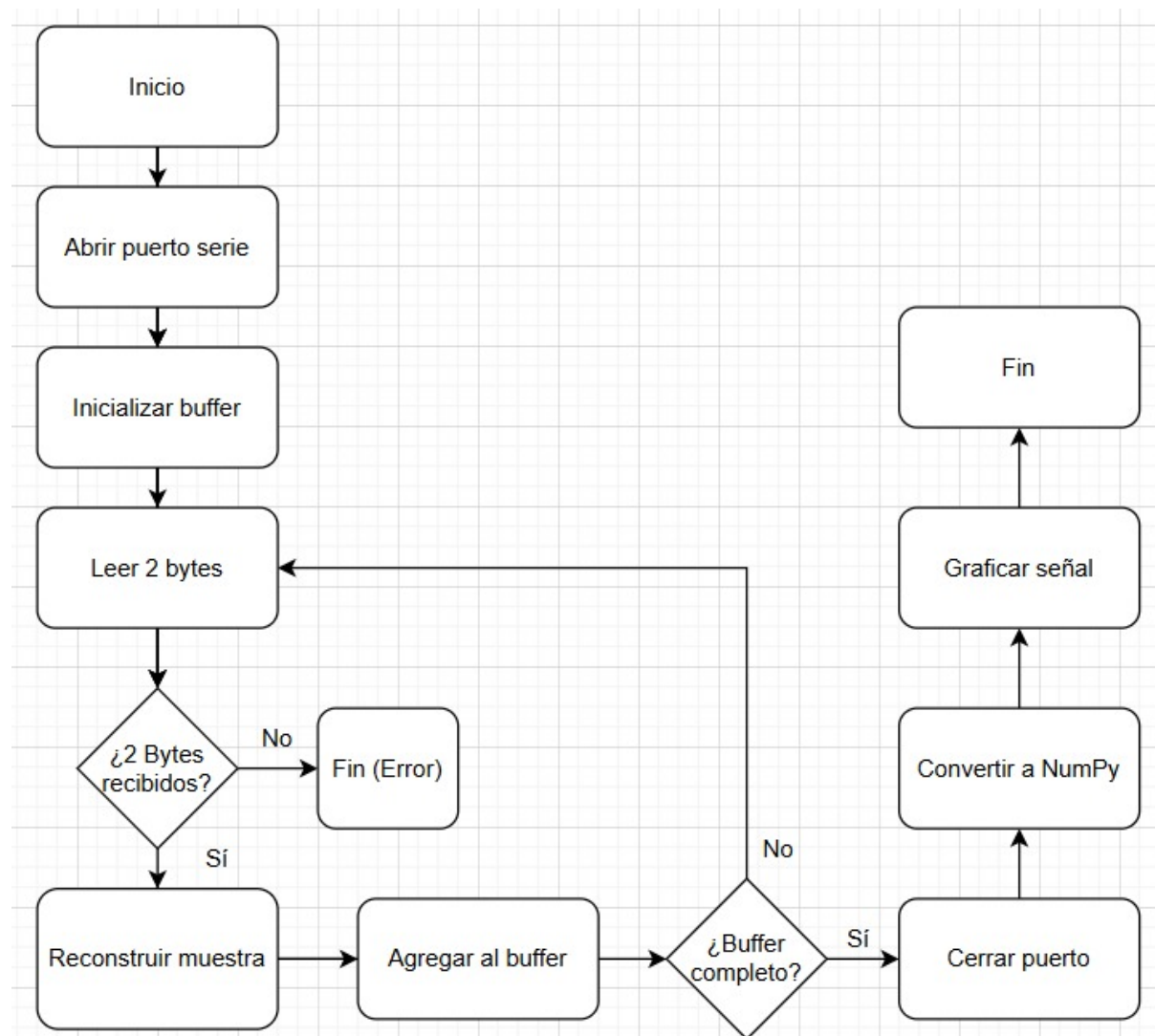


Figura 13: Diagrama de flujo del proceso de visualización de la señal

```
1 import serial
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 COM_PORT = 'COM5'
6 BAUDRATE = 9600
7 LISTSIZE = 12000
8
9 ser = serial.Serial('COM5', BAUDRATE, timeout=20)
10
11 print("Esperando datos de la LPC...")
12
13 data = []
14
15 for _ in range(LISTSIZE):
16     raw = ser.read(2)
17     if len(raw) < 2:
18         print("Error: no se recibieron suficientes datos")
```

```
19         break
20     sample = ((raw[0] & 0x03) << 8) | raw[1]
21     data.append(sample)
22
23 ser.close()
24 print("Datos recibidos. Graficando...")
25
26 data = np.array(data)
27
28 plt.figure(figsize=(12,4))
29 plt.plot(data)
30 plt.title("Senal grabada desde LPC")
31 plt.xlabel("Muestra")
32 plt.ylabel("Valor ADC")
33 plt.grid(True)
34 plt.show()
```

4.2. Firmware del proceso de adquisición de la señal de audio

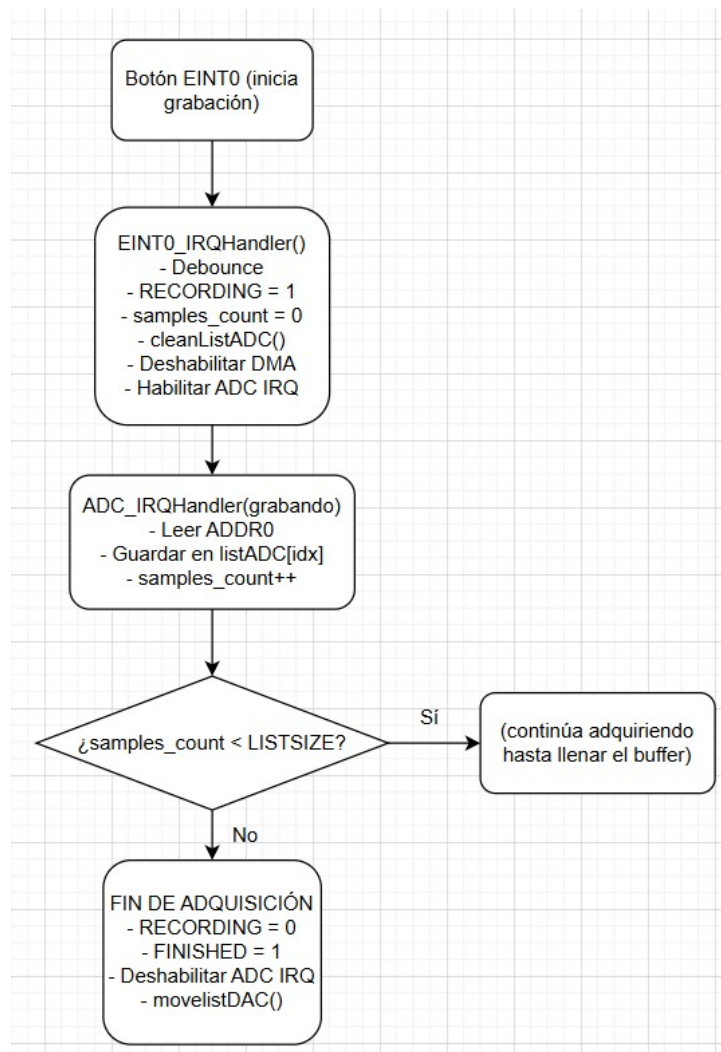


Figura 14: Diagrama de flujo del proceso de adquisición de la señal de audio

4.3. Firmware del SSE de Comunicación de datos

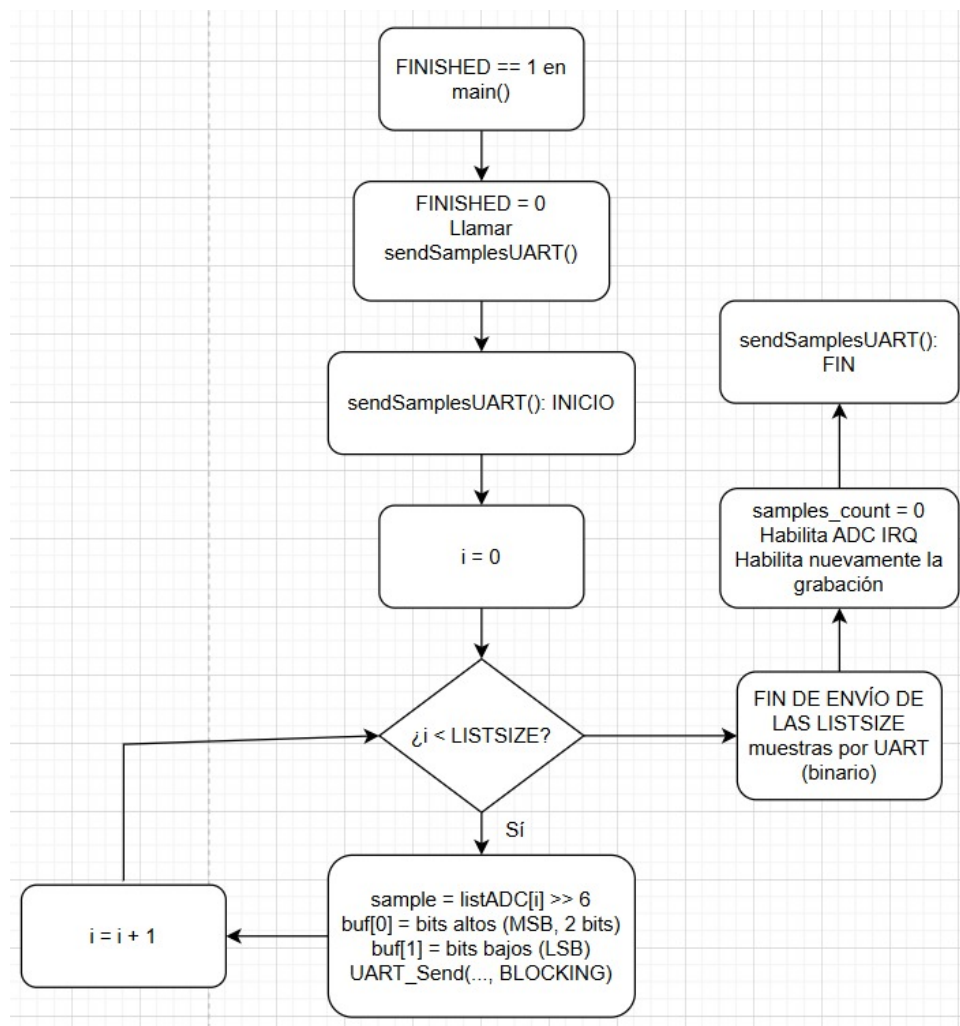


Figura 15: Diagrama de flujo del proceso de transmisión de datos

4.4. Firmware del SSE de Señalización óptica

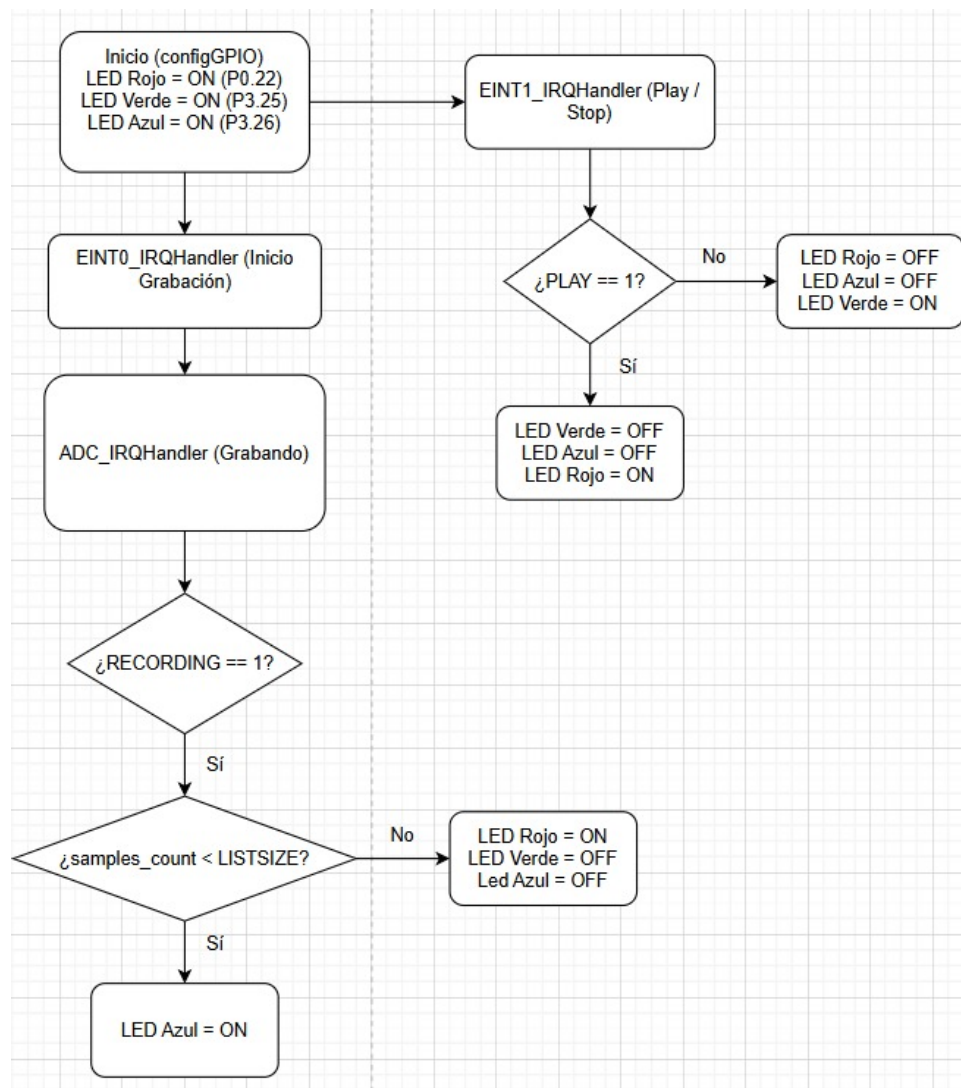


Figura 16: Diagrama de flujo del proceso de señalización óptica

4.5. Firmware del SSE de variación de velocidad

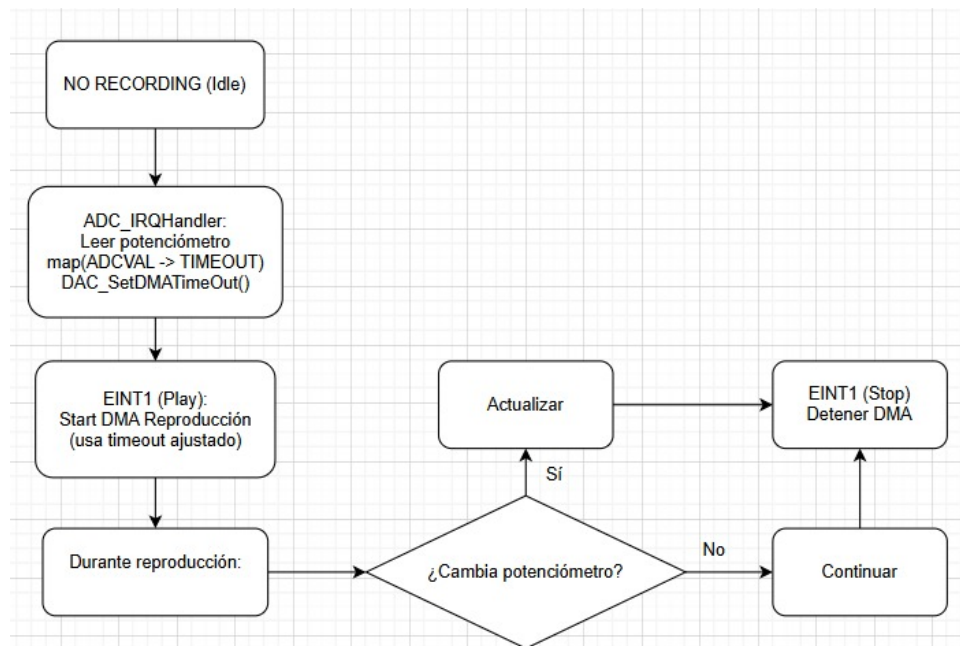


Figura 17: Diagrama de flujo del proceso de variación de velocidad de reproducción

4.6. Firmware del SSE de comienzo de grabación

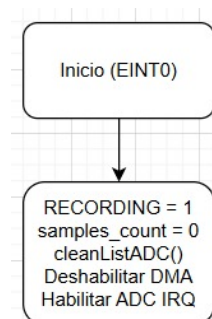


Figura 18: Diagrama de flujo del proceso de comienzo de grabación

4.7. Firmware del SSE para reanudar y detener la reproducción

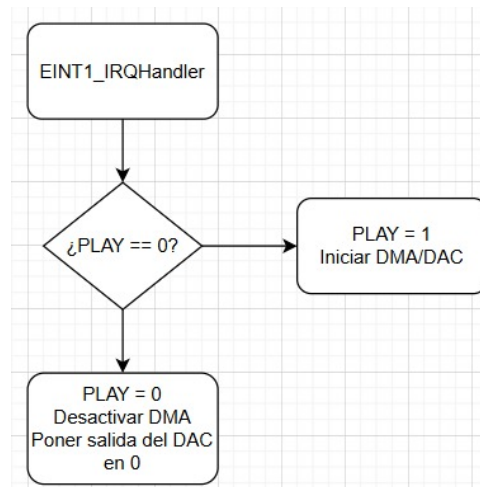


Figura 19: Diagrama de flujo del proceso para reanudar y detener la reproducción

5. Pruebas de sistema

N°	Descripción	Paso	Resultado Esperado	Resultado Obtenido	Observaciones
1	Prueba de inicio de grabación	Presionar botón REC con memoria vacía	Debe comenzar a grabar y aumentar el contador de muestras	PASA	
2	Prueba de límite de buffer	Llenar el buffer completo grabando hasta el máximo permitido	El sistema debe detener la grabación y activar bandera FINISHED	PASA	Ningún desbordamiento detectado
3	Prueba de calidad básica de audio	Hablar a 10–20 cm del micrófono durante la grabación	Las muestras deben presentar forma de onda reconocible y sin saturación	PASA	Audio claro

Cuadro 1: Prueba de Funcionalidad de Grabación

N°	Descripción	Paso	Resultado Esperado	Resultado Obtenido	Observaciones
1	Reproducción simple	Presionar PLAY luego de grabar	El audio debe reproducirse sin cortes	PASA	
2	Cambio de velocidad	Modificar parámetro de velocidad ($\uparrow$ o $\downarrow$) durante reproducción	La reproducción debe acelerarse o desacelerarse sin trabas	PASA	Ligerísimo ruido transitorio
3	Stop/Play	Presionar STOP durante reproducción y luego PLAY	Debe detenerse inmediatamente y reanudar desde el inicio	PASA	Pico de ruido blanco al reanudar y pausar

Cuadro 2: Pruebas de Reproducción

N°	Descripción	Paso	Resultado Esperado	Resultado Obtenido	Observaciones
1	Envío de muestras completo	Activar FI-NISHED y permitir envío binario vía UART	Deben enviarse los 12000 samples sin error	PASA	
2	Reinicio de captura luego del envío	Verificar que samples_count vuelva a cero	ADC debe quedar listo para nueva captura	PASA	

Cuadro 3: Pruebas de Comunicación UART

6. Conclusiones

El desarrollo del dispositivo grabador-reproductor permitió integrar módulos fundamentales de un sistema embebido: adquisición mediante ADC, temporización por interrupciones, reconstrucción de señal con DAC y verificación de datos vía UART. El sistema cumplió adecuadamente las funciones de captura, almacenamiento temporal y reproducción, mostrando un desempeño estable. Las pruebas realizadas confirmaron que el flujo completo de audio opera correctamente bajo diversas condiciones. Se observaron ruidos transitorios al alternar entre *play/pause* o al reiniciar buffers, efectos que pueden mitigarse con técnicas de silenciado o precarga. Como mejora futura, se propone incorporar un regulador *step-down* para simplificar la alimentación. En conjunto, el trabajo consolidó conocimientos de hardware y software embebido, demostrando la capacidad del microcontrolador para gestionar adquisición y reproducción de audio en tiempo real.

7. Bibliografía y referencias

Referencias

- [1] NXP Semiconductors, *LPC176x/5x User Manual*, Rev. 3.1 (2014).
- [2] Python Software Foundation, *PySerial Documentation*.
- [3] THAT Corporation, *Designing Microphone Preamplifiers*, AES 129th Convention (2010).
- [4] Sound-AU, *Low Noise Microphone Preamplifier*.

8. Anexo

8.1. Circuito esquemático completo

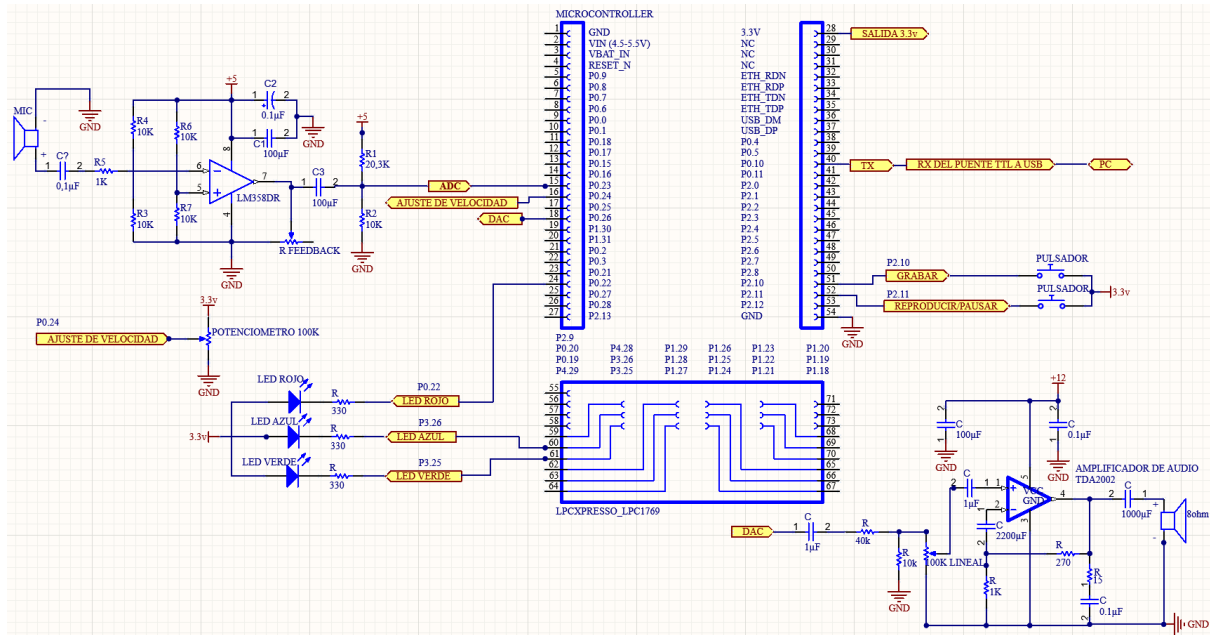


Figura 20: Circuito esquemático completo

8.2. Firmware del SEP

```
#include "LPC17xx.h"
#include "lpc_types.h"
#include "lpc17xx_adc.h"
#include "lpc17xx_dac.h"
#include "lpc17xx_gpdma.h"
#include "lpc17xx_pinsel.h"
#include "lpc17xx_exti.h"
#include "lpc17xx_uart.h"

#define ADC_RATE      8000
#define LISTSIZE      12000
#define TIMEOUT      7000
#define NUM_LISTS     3

void configADC(void);
void configDAC(void);
void configGPIO(void);
void configEINT0(void);
void configUART(void);
void configEINT1(void);
void configDMA(__IO uint16_t listADC[]);
void configNVIC(void);

uint32_t map(uint32_t x, uint32_t in_min, uint32_t in_max, uint32_t out_min,
             uint32_t out_max);
void cleanListADC(void);
void moveListDAC(void);
void buttonDebounce(void);
void sendSamplesUART(void);
```

```

30 /* Buffers y contadores */
31 __IO uint16_t listADC[LISTSIZE] = {0};
32 volatile uint32_t samples_count = 0; /* contador seguro (volatile) */
33
34 /* Flags */
35 uint8_t RECORDING = 0;
36 volatile uint8_t FINISHED = 0;
37
38 GPDMA_LLI_Type LLI_Array[NUM_LISTS];
39 GPDMA_Channel_CFG_Type dmaCFG;
40
41 /* ----- MAIN ----- */
42 int main(void)
43 {
44     configGPIO();
45     configEINT0();
46     configEINT1();
47     configADC();
48     configDAC();
49     configUART();
50     configNVIC();
51
52     while (1)
53     {
54         if (FINISHED)
55         {
56             /* marcaremos FINISHED = 0 antes de enviar para evitar reentradas */
57             FINISHED = 0;
58             sendSamplesUART();
59         }
60     }
61
62     return 0;
63 }
64
65 /* ----- funciones utilitarias ----- */
66 void cleanListADC(void)
67 {
68     for (uint32_t i = 0; i < LISTSIZE; i++) listADC[i] = 0;
69 }
70
71 void moveListDAC(void)
72 {
73     for (uint32_t i = 0; i < LISTSIZE; i++)
74     {
75         listADC[i] = listADC[i] << 6;
76     }
77 }
78
79 void buttonDebounce(void)
80 {
81     for (volatile uint32_t i = 0; i < 50000; i++) {}
82 }
83
84 uint32_t map(uint32_t x, uint32_t in_min, uint32_t in_max, uint32_t out_min,
85             uint32_t out_max)
86 {
87     return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min;
88 }
89
90 /* ----- configuraciones ----- */
91 void configGPIO(void)

```



```

91 {
92     PINSEL_CFG_Type pinCFG;
93     /* P0.23 AD0.0 */
94     pinCFG.Funcnum = PINSEL_FUNC_1; pinCFG.OpenDrain = PINSEL_PINMODE_NORMAL;
95     pinCFG.Pinmode = PINSEL_PINMODE_TRISTATE; pinCFG.Pinnum = PINSEL_PIN_23;
96     pinCFG.Portnum = PINSEL_PORT_0;
97     PINSEL_ConfigPin(&pinCFG);
98
99     /* P0.24 AD0.1 */
100     pinCFG.Pinnum = PINSEL_PIN_24; PINSEL_ConfigPin(&pinCFG);
101
102     /* P2.10 EINT0 */
103     pinCFG.Funcnum = PINSEL_FUNC_1; pinCFG.Pinmode = PINSEL_PINMODE_PULLDOWN;
104     pinCFG.Pinnum = PINSEL_PIN_10; pinCFG.Portnum = PINSEL_PORT_2;
105     PINSEL_ConfigPin(&pinCFG);
106
107     /* P2.11 EINT1 */
108     pinCFG.Pinnum = PINSEL_PIN_11; PINSEL_ConfigPin(&pinCFG);
109
110     /* P0.26 DAC */
111     pinCFG.Funcnum = PINSEL_FUNC_2; pinCFG.Pinmode = PINSEL_PINMODE_TRISTATE;
112     pinCFG.Pinnum = PINSEL_PIN_26; pinCFG.Portnum = PINSEL_PORT_0;
113     PINSEL_ConfigPin(&pinCFG);
114
115     /* LEDs */
116     LPC_GPIO0->FIODIR |= (1 << 22);
117     LPC_GPIO3->FIODIR |= (1 << 25);
118     LPC_GPIO3->FIODIR |= (1 << 26);
119     LPC_GPIO0->FIOSET |= (1 << 22);
120     LPC_GPIO3->FIOSET |= (1 << 25);
121     LPC_GPIO3->FIOSET |= (1 << 26);
122
123     /* UART2 pins P0.10 (TX) / P0.11 (RX) */
124     pinCFG.Funcnum = 1; pinCFG.OpenDrain = 0; pinCFG.Pinmode = 0;
125     pinCFG.Pinnum = 10; pinCFG.Portnum = 0; PINSEL_ConfigPin(&pinCFG);
126     pinCFG.Pinnum = 11; PINSEL_ConfigPin(&pinCFG);
127 }
128
129 void configADC(void)
130 {
131     ADC_Init(LPC_ADC, ADC_RATE);
132     ADC_ChannelCmd(LPC_ADC, 0, ENABLE);
133     ADC_ChannelCmd(LPC_ADC, 1, ENABLE);
134     ADC_BurstCmd(LPC_ADC, ENABLE);
135     /* habilita interrupcion por finalizacion del canal 0 */
136     ADC_IntConfig(LPC_ADC, ADC_ADINTENO, ENABLE);
137     ADC_StartCmd(LPC_ADC, ADC_START_CONTINUOUS);
138 }
139
140 void configDAC(void)
141 {
142     DAC_CONVERTER_CFG_Type dacCFG;
143     dacCFG.CNT_ENA = SET;
144     dacCFG.DMA_ENA = SET;
145     DAC_SetDMATimeOut(LPC_DAC, TIMEOUT);
146     DAC_ConfigDAConverterControl(LPC_DAC, &dacCFG);
147     DAC_Init(LPC_DAC);
148 }
149
150 void configDMA(__IO uint16_t listADC[])
151 {
152     for (int i = 0; i < NUM_LISTS; i++)
153     {

```

```

151     LLI_Array[i].DstAddr = (uint32_t)&(LPC_DAC->DACR);
152     LLI_Array[i].SrcAddr = (uint32_t)(listADC + i * 4095);
153
154     if (i == (NUM_LISTS - 1))
155         LLI_Array[i].NextLLI = (uint32_t)&LLI_Array[0];
156     else
157         LLI_Array[i].NextLLI = (uint32_t)&LLI_Array[i + 1];
158
159     LLI_Array[i].Control = 4095
160         | (1 << 18) /* source width 16 bit */
161         | (1 << 22) /* dest width = word 32 bits */
162         | (1 << 26) /* source increment */
163         ;
164 }
165
166 dmaCFG.ChannelNum = 0;
167 dmaCFG.TransferSize = 4095;
168 dmaCFG.TransferWidth = 0;
169 dmaCFG.TransferType = GPDMA_TRANSFERTYPE_M2P;
170 dmaCFG.SrcConn = 0;
171 dmaCFG.DstConn = GPDMA_CONN_DAC;
172 dmaCFG.SrcMemAddr = (uint32_t)listADC;
173 dmaCFG.DstMemAddr = 0;
174 dmaCFG.DMALLI = (uint32_t)&LLI_Array[0];
175
176 GPDMA_Init();
177 GPDMA_Setup(&dmaCFG);
178 GPDMA_ChannelCmd(0, ENABLE);
179 }
180
181 void configEINT0(void)
182 {
183     EXTI_InitTypeDef exti;
184     exti.EXTI_Mode = EXTI_MODE_EDGE_SENSITIVE;
185     exti.EXTI_polarity = EXTI_POLARITY_HIGH_ACTIVE_OR_RISING_EDGE;
186     exti.EXTI_Line = EXTI_EINT0;
187     EXTI_Config(&exti);
188 }
189
190 void configEINT1(void)
191 {
192     EXTI_InitTypeDef exti;
193     exti.EXTI_Mode = EXTI_MODE_EDGE_SENSITIVE;
194     exti.EXTI_polarity = EXTI_POLARITY_HIGH_ACTIVE_OR_RISING_EDGE;
195     exti.EXTI_Line = EXTI_EINT1;
196     EXTI_Config(&exti);
197 }
198
199 void configNVIC(void)
200 {
201     NVIC_EnableIRQ(ADC_IRQn);
202
203     EXTI_ClearEXTIFlag(EXTI_EINT0);
204     NVIC_EnableIRQ(EINT0_IRQn);
205
206     EXTI_ClearEXTIFlag(EXTI_EINT1);
207     NVIC_EnableIRQ(EINT1_IRQn);
208
209     /* GPDMA inicialmente deshabilitado */
210     GPDMA_ChannelCmd(0, DISABLE);
211 }
212
213 void configUART(void)

```

```

214 {
215     UART_CFG_Type      UARTConfigStruct;
216     UART_FIFO_CFG_Type UARTFIFOConfigStruct;
217
218
219     UART_ConfigStructInit(&UARTConfigStruct); /* 9600-8N1 por defecto */
220     UART_Init(LPC_UART2, &UARTConfigStruct);
221
222     UART_FIFOConfigStructInit(&UARTFIFOConfigStruct);
223     UART_FIFOConfig(LPC_UART2, &UARTFIFOConfigStruct);
224
225     UART_TxCmd(LPC_UART2, ENABLE);
226     UART_IntConfig(LPC_UART2, UART_INTCFG_RBR, DISABLE);
227     UART_IntConfig(LPC_UART2, UART_INTCFG_RLS, DISABLE);
228 }
229
230 /* ----- INTERRUPT HANDLERS ----- */
231 void ADC_IRQHandler(void)
232 {
233     static uint32_t ADCVAL = 0;
234     static uint32_t ADCVALMAP = 0;
235
236     if (RECORDING)
237     {
238         /* grabamos mientras no superemos LISTSIZE */
239         if (samples_count < LISTSIZE)
240         {
241             LPC_GPIO3->FIOCLR |= (1 << 26); /* Led azul ON */
242             listADC[samples_count] = ((LPC_ADC->ADDR0) >> 6) & 0x3FF;
243             samples_count++;
244         }
245
246         if (samples_count >= LISTSIZE)
247         {
248             /* terminamos la grabacion */
249             LPC_GPIO0->FIOSET |= (1 << 22);
250             LPC_GPIO3->FIOSET |= (1 << 25);
251             LPC_GPIO3->FIOSET |= (1 << 26);
252
253             RECORDING = 0;
254             FINISHED = 1;
255
256             /* deshabilitar NVIC del ADC para evitar sobrescrituras mientras
257             procesamos */
258             NVIC_DisableIRQ(ADC_IRQn);
259
260             /* preparar datos para DAC */
261             moveListDAC();
262         }
263     }
264     else /* no recording: ajustar DAC timeout segun potenciometro */
265     {
266         ADCVAL = ((LPC_ADC->ADDR1) >> 6) & 0x3FF;
267         ADCVALMAP = map(ADCVAL, 0, 1024, 5000, 20000);
268         DAC_SetDMATimeOut(LPC_DAC, ADCVALMAP);
269     }
270
271     /* limpiar flag de ADC (lectura repetida para limpiar) */
272     LPC_ADC->ADGDR &= LPC_ADC->ADGDR;
273 }
274 void EINT0_IRQHandler(void)
275 {

```

```

276     buttonDebounce();
277
278     RECORDING = 1;
279     samples_count = 0;
280     cleanListADC();
281
282     /* apagar canales DMA y preparar */
283     GPDMA_ChannelCmd(0, DISABLE);
284
285     NVIC_EnableIRQ(ADC_IRQn);
286     EXTI_ClearEXTIFlag(EXTI_EINT0);
287 }
288
289 void EINT1_IRQHandler(void)
290 {
291     static uint8_t PLAY = 0;
292     buttonDebounce();
293
294     if (PLAY)
295     {
296         LPC_GPIO3->FIOSET |= (1 << 25);
297         LPC_GPIO3->FIOSET |= (1 << 26);
298         LPC_GPIO0->FIOCLR |= (1 << 22);
299         GPDMA_ChannelCmd(0, DISABLE);
300         DAC_UpdateValue(LPC_DAC, 0);
301         PLAY = 0;
302     }
303     else
304     {
305         LPC_GPIO0->FIOSET |= (1 << 22);
306         LPC_GPIO3->FIOSET |= (1 << 26);
307         LPC_GPIO3->FIOCLR |= (1 << 25);
308         configDMA(listADC);
309         PLAY = 1;
310     }
311
312     EXTI_ClearEXTIFlag(EXTI_EINT1);
313 }
314
315 void sendSamplesUART(void)
316 {
317     /* Envio binario: 2 bytes por muestra (MSB, LSB) con UART_Send BLOCKING */
318     uint8_t buf[2];
319
320     for (uint32_t i = 0; i < LISTSIZE; i++)
321     {
322         uint16_t sample = listADC[i] >> 6; /* 10 bits: 0..1023 */
323         buf[0] = (uint8_t)((sample >> 8) & 0x03); /* dos bits altos */
324         buf[1] = (uint8_t)(sample & 0xFF);
325         UART_Send(LPC_UART2, buf, 2, BLOCKING);
326     }
327
328     /* una vez enviado, re-habilitamos el ADC si queremos grabar otra vez */
329     samples_count = 0;
330     /* volver a permitir ADC IRQ si se desea */
331     NVIC_EnableIRQ(ADC_IRQn);
332 }

```

Listing 1: Código C principal para LPC1769

8.3. Lista de materiales

- 2 pulsadores
- Potenciómetro de 100 k Ω
- Amplificador de audio (Trabajo Práctico de la asignatura Tecnología Electrónica)
- Micrófono electret
- Amplificador LM358
- Capacitores:
 - 0,1 μF $\times$ 2
 - 22 μF
 - 100 μF
 - 1 μF
- Resistores:
 - 10 k Ω $\times$ 3
 - 1 k Ω
 - Trimpot 30 k Ω
 - Trimpot 10 k Ω
 - 333 Ω $\times$ 3
- LED RGB
- Placa perforada 9 $\times$ 15 cm
- Punte USB to TTL
- LPC1769
- Jack 3.5 mm hembra
- Tira de pines hembra 2.54 mm